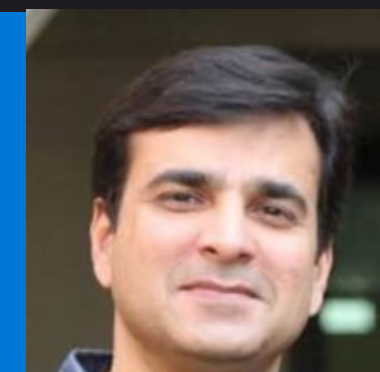


Introduction to Java

Absolute Basics



Zubair Ahmed

Assistant Professor
SZABIST Islamabad

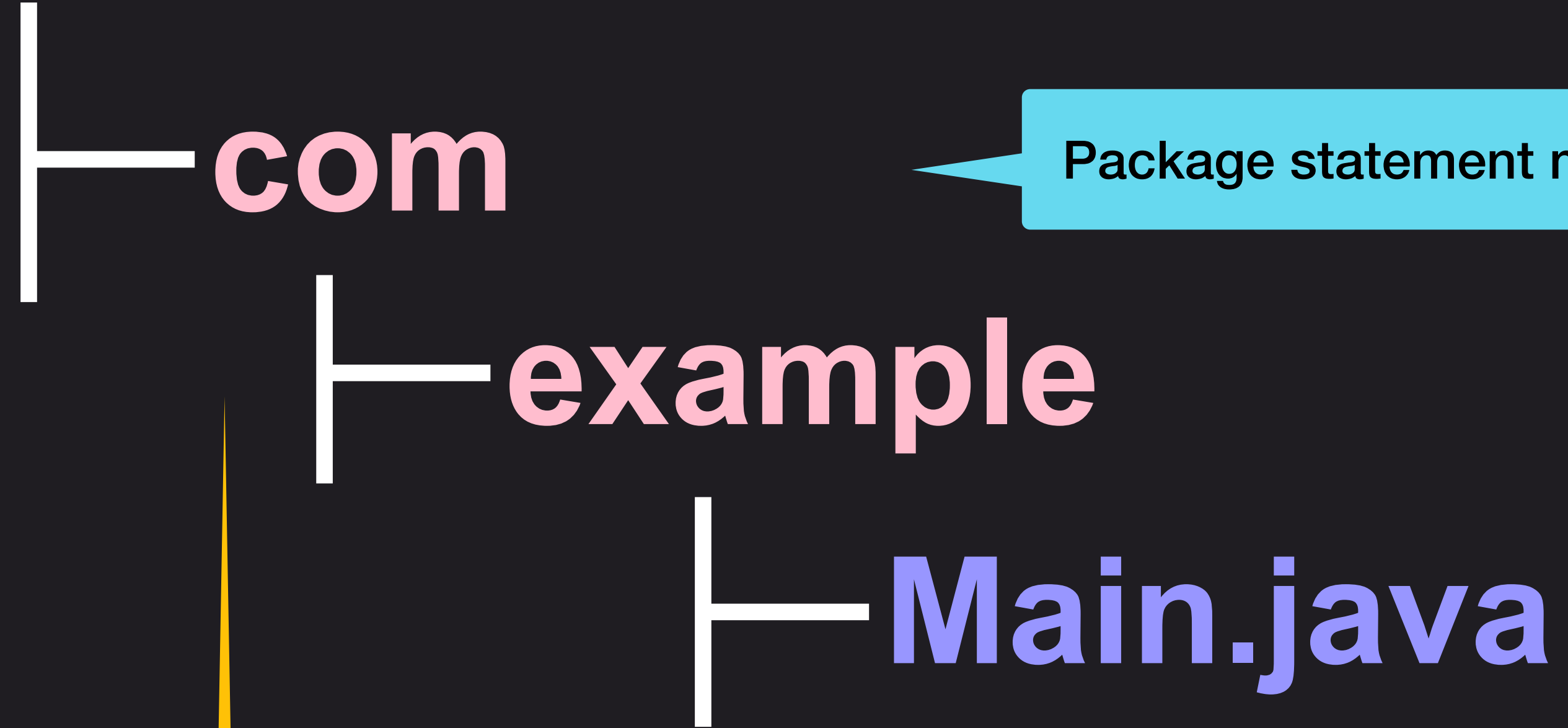
```
package com.example; 1
```

```
import java.lang.*; 2
```

Statements should be exactly as they appear below

```
public class Main { 3
    public static void main(String[] args) {
        System.out.println("Hi");
    }
}
```

MyProject/



Package statement means we have a folders hierarchy

com is the parent and example is child directory where Main.java resides

```
package com.example;  
  
public class Main {  
  
}
```

Convention is to reverse the domain name in order to come up with the package name

Which class can run on its own?

Because it has the main method

		Main	
		Main()	
		main(String[]) void	

		Person
		Person()

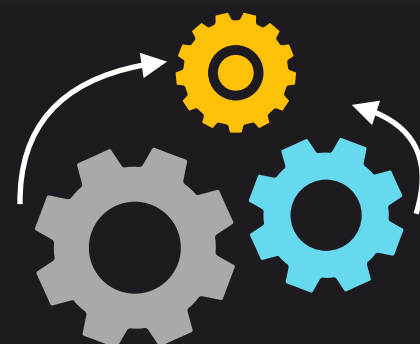
Step 1



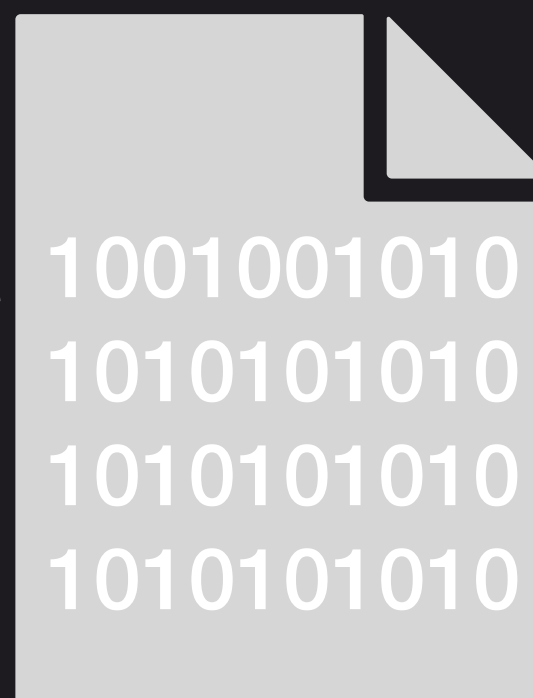
Step 2

COMPILATION

JAVA COMPILER



Step 3



.class file
(byte code)

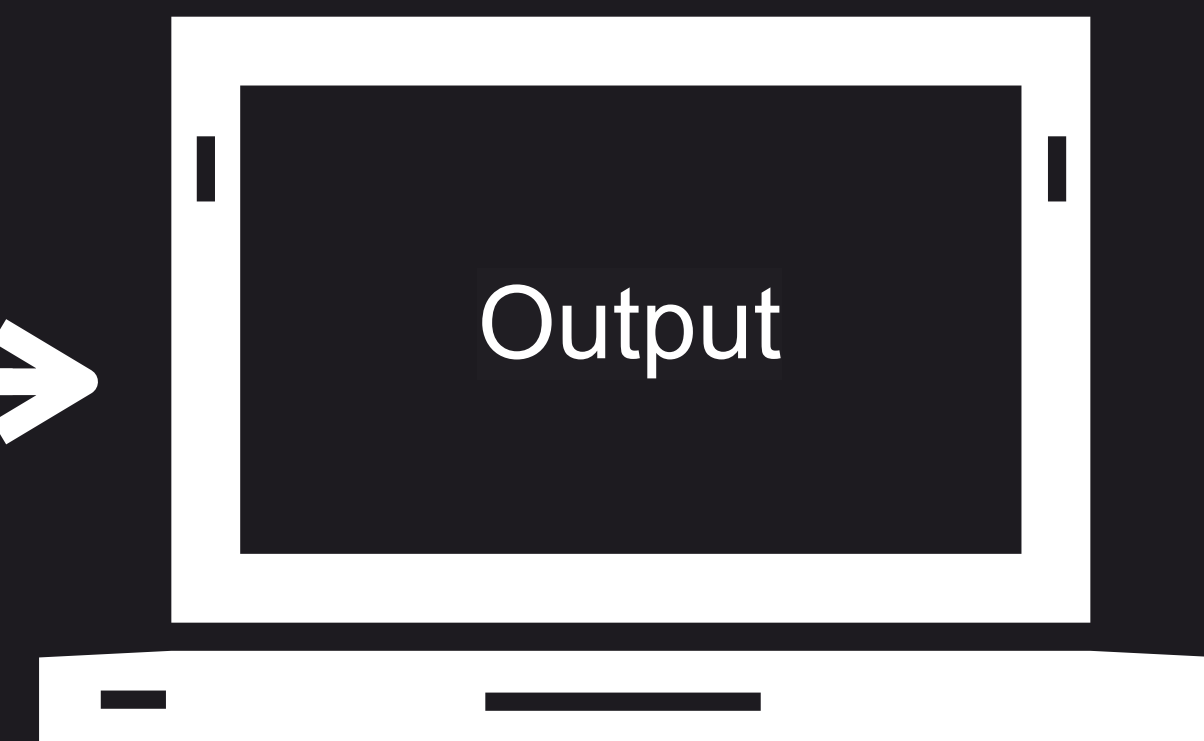
Step 3

EXECUTION

JAVA VIRTUAL MACHINE



Step 4



Compile & Run



```
$ javac Main.java
```

Compile & Run



```
$ java Main
```

Compile & Run



```
$ javac Main.java
```

```
$ java Main
```

```
Hi
```

```
package com.example;
```

```
import java.lang.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hi " + args[0] + args[0]);  
    }  
}
```

The String Array can be used to send arguments to your class

Compile & Run



```
$ javac Main.java
```

```
$ java Main Zubair
```

```
Hi Zubair
```

If you miss send args

An exception will be thrown

No Command-line args



```
$ java Main
```

```
package com.example;
```

```
import java.lang.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hi "+args[0]);  
    }  
}
```

The String Array can be used to send arguments to your class

```
$ java Main
```

Trying to run the program with command line args

```
/Library/Java/JavaVirtualMachines/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/1  
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0  
    at com.example.Main.main(Main.java:8)
```

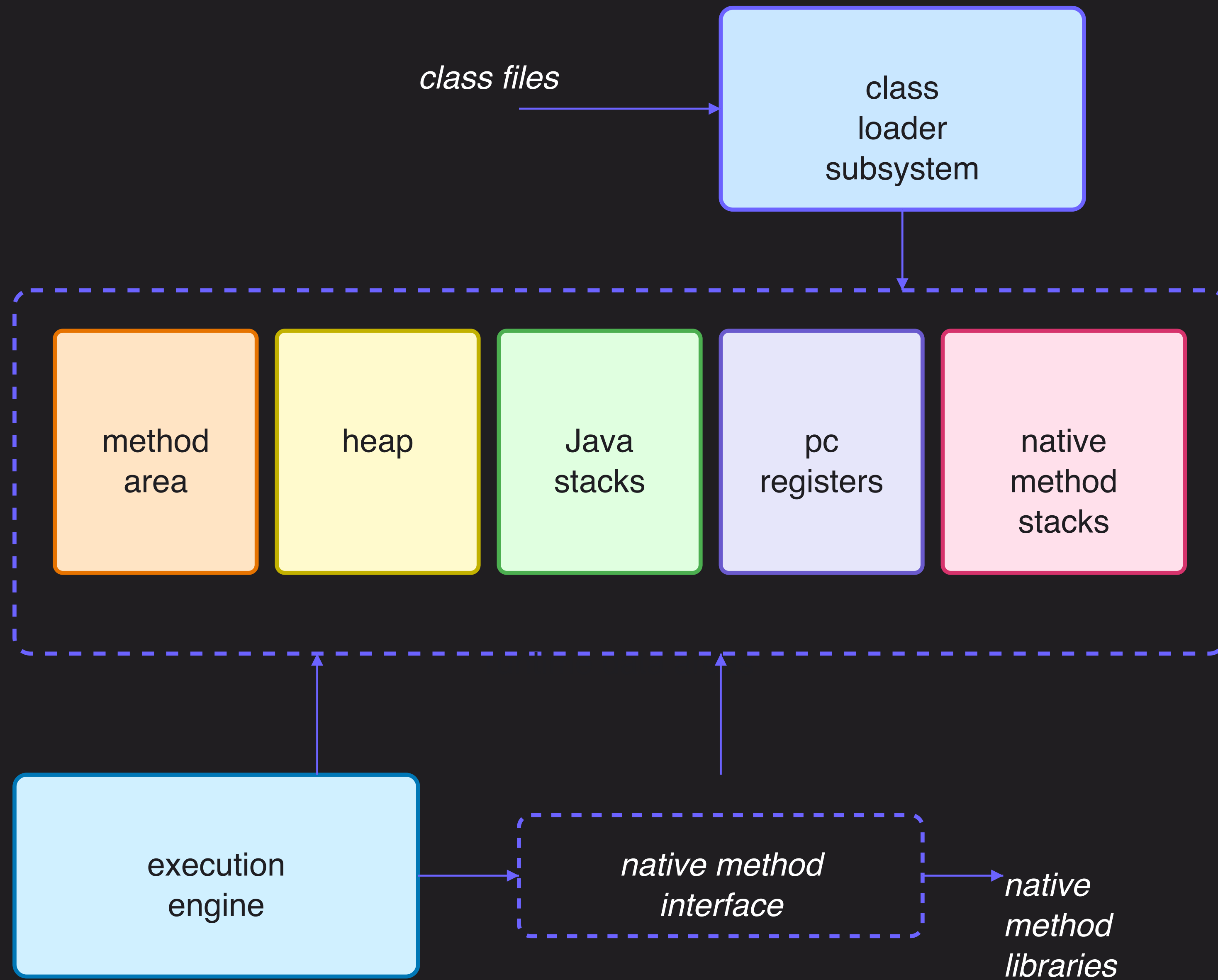
```
Process finished with exit code 1
```

**What happens when you
run your java class?**

IDE executes your program

- IDE creates a new process to execute `/bin/java`
- `java` executable boots up the JVM inside the process
- Initializes subsystems (GC, JIT Compiler, class loader, etc.)
- Loads the bootstrap classes i.e. `java.lang.Object`, `java.lang.String`
- Create the main thread, the thread which will run the main method
- After finishing main method thread exits by calling `system.exit()`
- An exit code of (0) is returned for normal else non-zero value is returned

JVM Internals



Example Class

```
package com.example;
```

```
public class Person {  
    //absolutely nothing inside the class  
}
```

Object Creation

```
package com.example;  
import java.lang.*;  
public class Main {  
    public static void main(String[] args) {
```

The class you create behaves like a data type

```
        Person p = new Person();  
        System.out.println(p);  
    }  
}
```

We need new keyword to create an object of a class

Printing the object reference

>_ com.example.Person@8efb846

Object Creation

```
package com.example;  
import java.lang.*;  
public class Main {  
    public static void main(String[] args) {
```

The class you create behaves like a data type

```
        Person p = new Person();  
        System.out.println(p.toString());  
    }  
}
```

We need new keyword to create an object of a class

Printing the object reference

```
>_ com.example.Person@8efb846
```

To create an object

- You need a constructor
- You have to use `new` keyword

What gets returned is a pointer to an object on the heap called `Object reference`

Note: heap is one of the memory areas JVM uses

Object Reference's Type



Either itself e.g. **Person**



Or the **parent** of the **Person** class

Parent of Person class

```
package com.example;
```

Nothing mentioned here

```
public class Person {
```

```
//absolutely nothing inside the class
```

```
}
```

In java we use extends keyword to mention a parent of a class

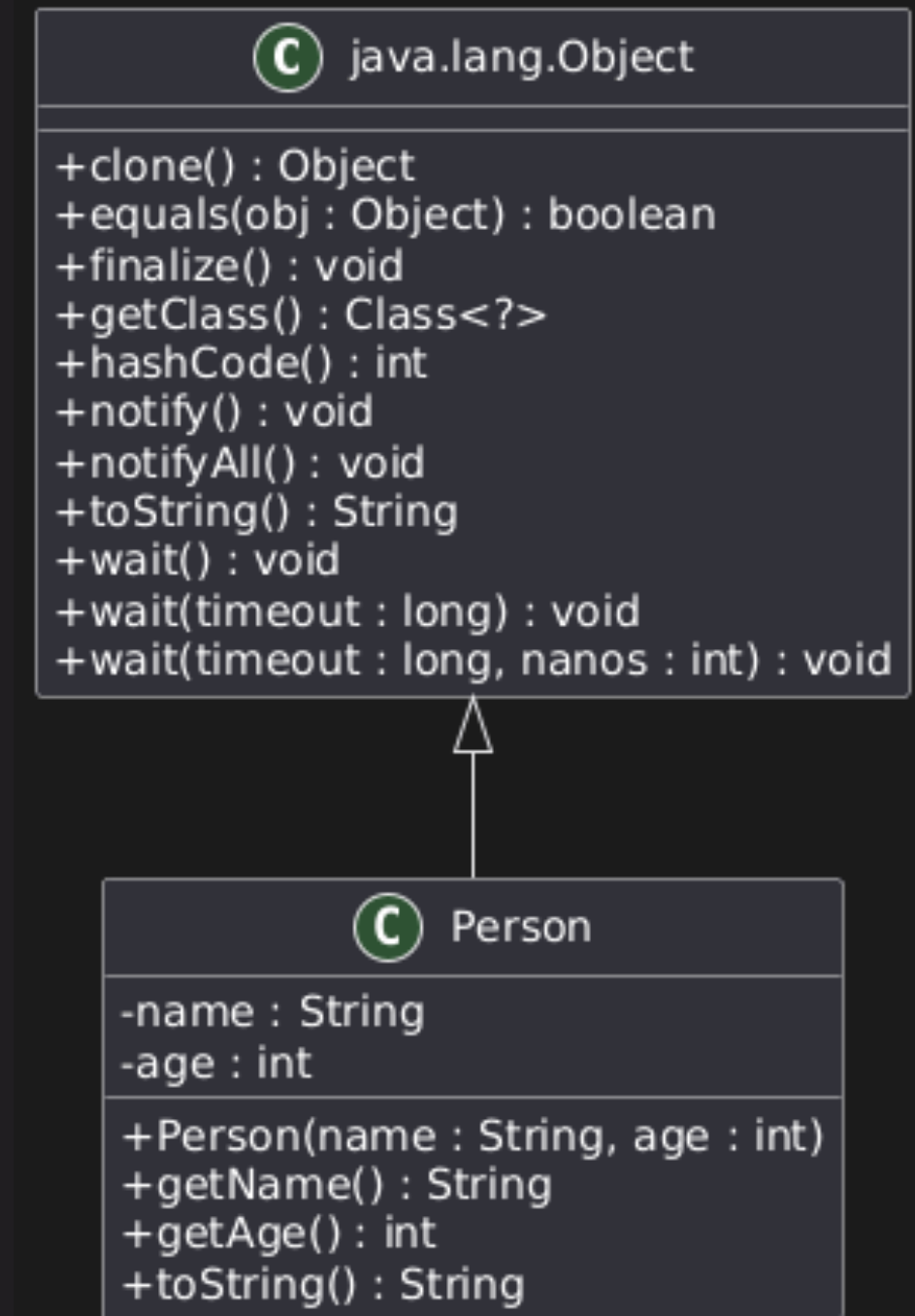
Parent of Person class

```
package com.example;
```

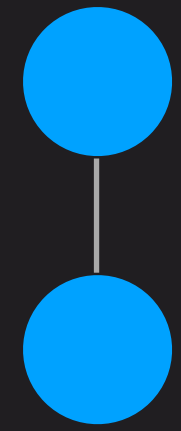
Implicitly written for you

```
public class Person extends Object{  
    //absolutely nothing inside the class  
}
```

Every class in java has a parent either you write it explicitly or not



Object Reference's Type



Either itself e.g. **Person**

Or the **parent** of the **Person** keyword

You can either decide type of Person
you created to be of Person type

```
Person p = new Person();
```

Object Reference's Type

- Either itself e.g. `Person`
- Or the `parent` of the `Person` keyword

Or you can decide it to be of its parent type i.e. `java.lang.Object`

```
Object p = new Person();
```

Note: You will not be able to call methods of the person class that are not in the Object class!

Exercise

```
package com.example;  
import java.lang.*;  
public class Main {  
    public static void main(String[] args) {  
  
        Person p = new Person();  
        System.out.println(p);  
    }  
}
```

Modify the Person class given in slide 16 to println your name rather gibberish com.example.Person@8efb846 from the above given Main class

Solution

```
package com.example;  
  
public class Person{  
    public String toString() {  
        return "Zubair";  
    }  
}
```

Why inheritance?

Student

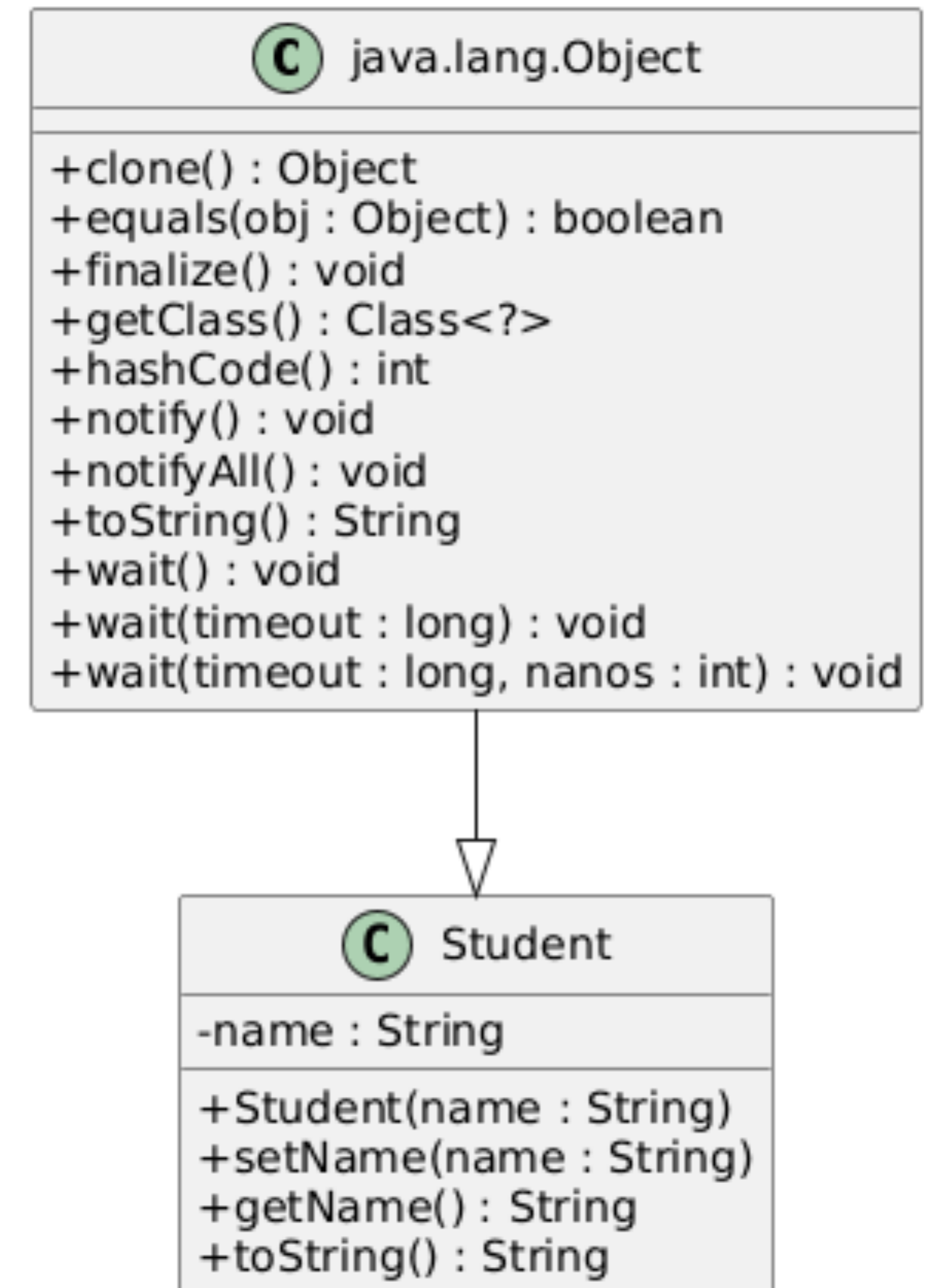
```
package com.example;

public class Student {
    private String name;

    Student(String name) {
        this.name = name;
    }

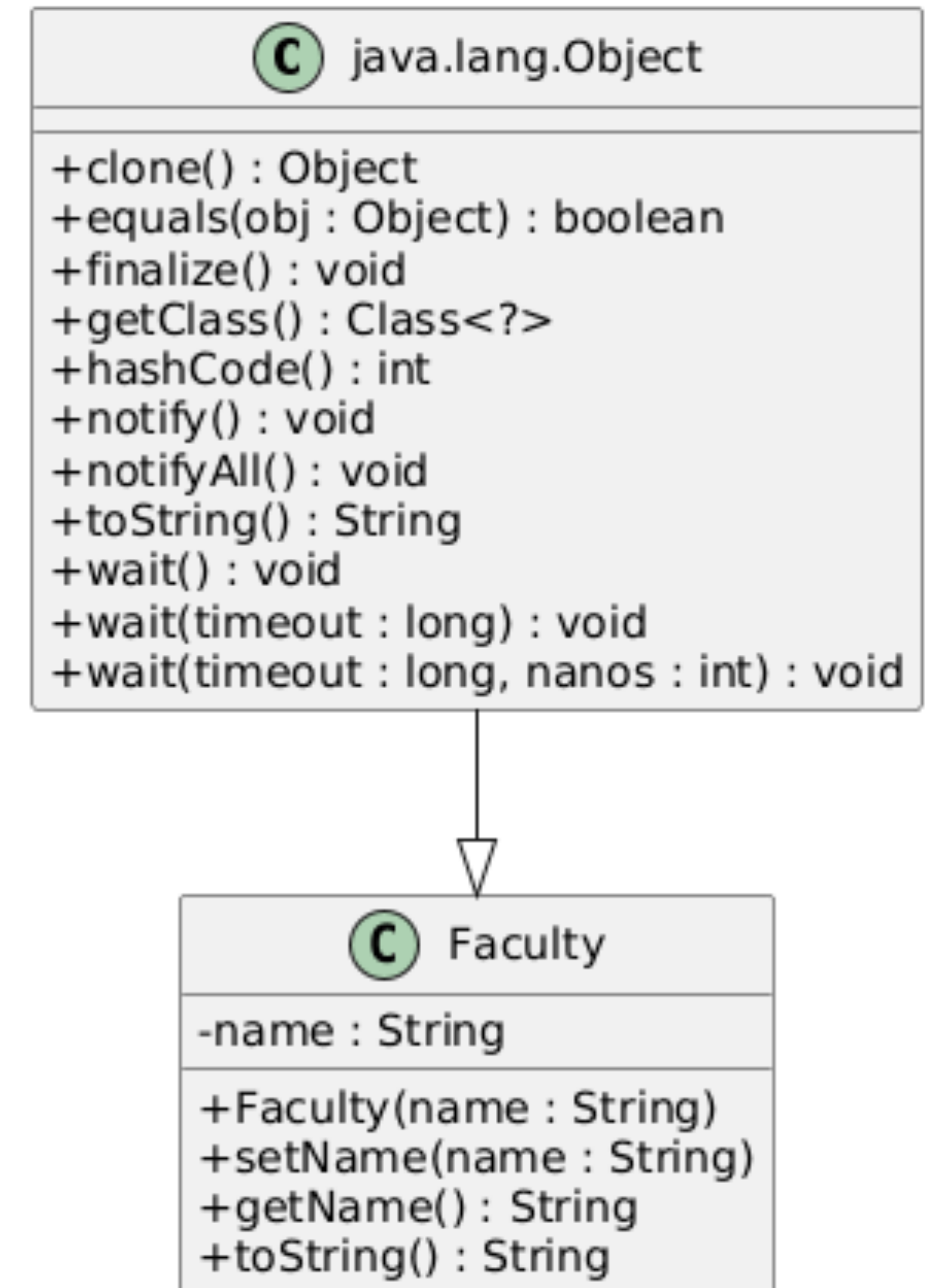
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

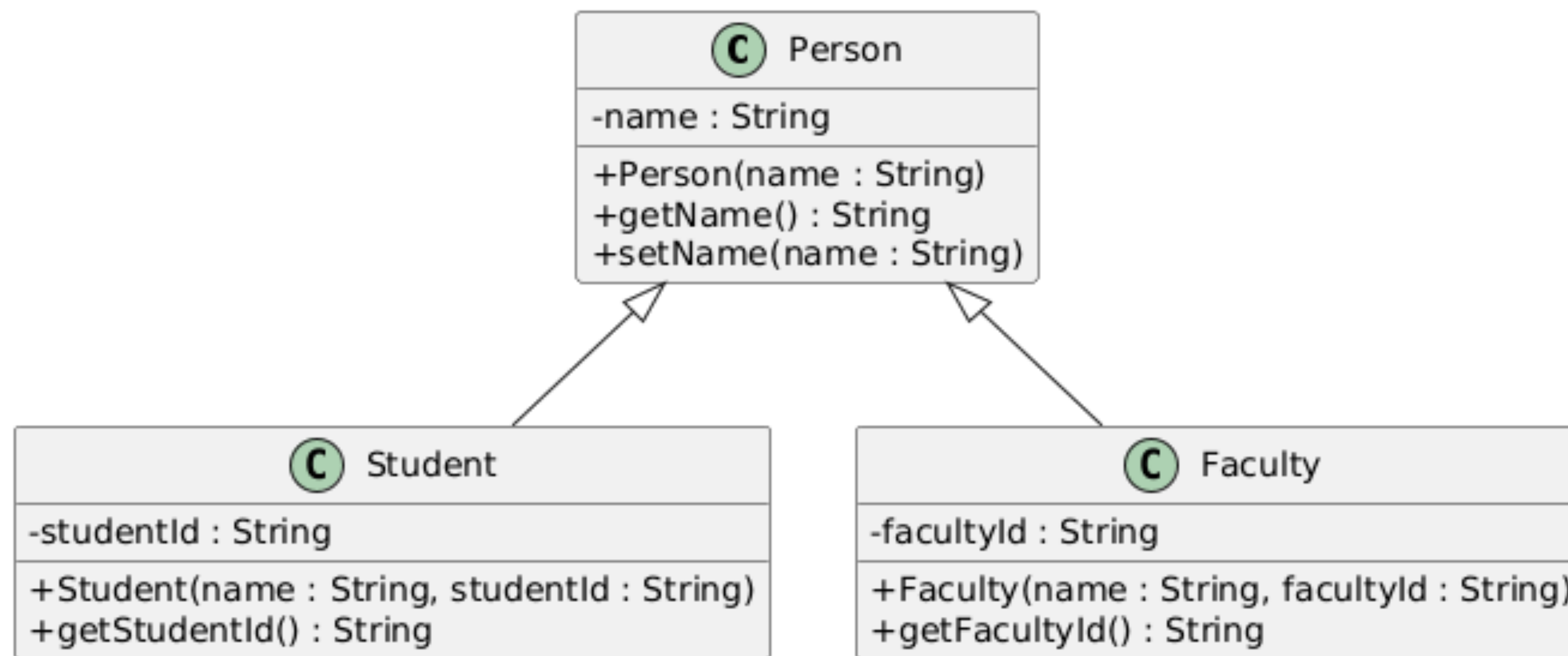


Faculty

```
public class Faculty {  
    private String name;  
  
    Faculty(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```



**We need a parent to host
common fields**



```
public class Person{
    private String name;

    public Person(String name) {
        this.name = name;
    }
}

public class Student extends Person {
    Student(String name) {
        super(name);
    }
}

public class Faculty extends Person {
    Faculty(String name) {
        super(name);
    }
}
```

The code shows the implementation of the classes. The **Person** class is at the top, with a private `String` attribute `name` and a public constructor `Person(String name)` that initializes `this.name`. Below it are the **Student** and **Faculty** classes, both extending **Person**. The **Student** class has a public constructor `Student(String name)` that calls `super(name)`. The **Faculty** class has a public constructor `Faculty(String name)` that also calls `super(name)`. Arrows indicate the inheritance relationship from the code to the diagram above.

```
public class Person{  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
  
}
```

```
public class Student extends Person {  
    Student(String name) {  
        super(name);  
    }  
}  
  
public class Faculty extends Person {  
    Faculty(String name) {  
        super(name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Faculty f = new Faculty("Zubair");  
        Student s = new Student("Ali");  
    }  
}
```

```
public class Person{  
    private String name;
```

```
    public Person(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

```
}
```

```
public class Student extends Person {  
    Student(String name) {  
        super(name);  
    }  
}
```

```
public class Faculty extends Person {  
    Faculty(String name) {  
        super(name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Faculty f = new Faculty("Zubair");  
        Student s = new Student("Ali");  
    }  
}
```

```
        public class Person{
            private String name;

            public Person(String name) {
                this.name = name;
            }
            public String getName() {
                return name;
            }
            public void setName(String name) {
                this.name = name;
            }
        }

public class Student extends Person {
    Student(String name) {
        super(name);
    }
}

public class Faculty extends Person {
    Faculty(String name) {
        super(name);
    }
}

public class Main {
    public static void main(String[] args) {
        Faculty f = new Faculty("Zubair");
        Student s = new Student("Ali");
    }
}
```

```
public class Person{
    private String name;

    public Person(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

```
public class Student extends Person {
    Student(String name) {
        super(name);
    }
}
```

```
public class Faculty extends Person {
    Faculty(String name) {
        super(name);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Faculty f = new Faculty("Zubair");
        Student s = new Student("Ali");
        System.out.println(f.getName());
    }
}
```

```
public class Person{
    private String name;

    public Person(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

```
public class Student extends Person {
    Student(String name) {
        super(name);
    }
}
```

```
public class Faculty extends Person {
    Faculty(String name) {
        super(name);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Faculty f = new Faculty("Zubair");
        Student s = new Student("Ali");
        System.out.println(f.getName());
        System.out.println(s.getName());
    }
}
```

Abstract doesn't let a class to be instantiate

```
public abstract class Person{  
    private String name;
```

```
    public Person(String name) {  
        this.name = name;
```

```
    }  
    public String getName() {  
        return name;
```

```
    }  
    public void setName(String name) {  
        this.name = name;
```

```
    }
```

```
}
```

```
public class Student extends Person {  
    Student(String name) {  
        super(name);  
    }  
}
```

```
public class Faculty extends Person {  
    Faculty(String name) {  
        super(name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Faculty f = new Faculty("Zubair");  
        Student s = new Student("Ali");  
        System.out.println(f.getName());  
        System.out.println(s.getName());  
    }  
}
```



```
new Person("Zubair");
```

Abstract classes

Uses of Abstract ?

Since you cannot create an object of Abstract class, they are useful for forcing certain behaviour over their child classes.

Abstract class:Example

```
public class Employee {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Abstract class:Example

```
public class Employee {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
    public double earnings();  
}
```

An abstract method will not have a body

Abstract class:Example

```
public class Employee {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
    public abstract double earnings();  
}
```

An abstract method will not have a body

Also you have to declare it with abstract keyword

Abstract class:Example

If a class has one abstract method, it must be declared with abstract keyword

```
public abstract class Employee {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
    public abstract double earnings();  
}
```

An abstract method will not have a body

Also you have to declare it with abstract keyword

Extend an Abstract class

```
public class SalariedEmployee extends Employee{  
    private double weeklySalary;  
  
    @Override  
    public double earnings() {  
        return weeklySalary;  
    }  
}
```

You must declare the abstract method
of your parent

As an exercise create an object with employee name & his salary.

Another Employee Type

```
public class HourlyEmployee extends Employee{
    private double rate;
    private double wage;
    public HourlyEmployee(String name, double rate, double wage) {
        super(name);
        this.rate = hourlyRate;
        this.wage = hourlyWage;
    }
    @Override
    public double earnings() {
        return rate * wage;
    }
}
```

As an exercise create an object with employee name & his salary.

Abstract can still behave like Types

Although we are restricted by object creation, we can create Arrays or Lists to store child classes of Abstract classes.

Exercise

Create an array of Employee (abstract class) and store objects of Salaried and Hourly employees in it.

Exercise

Create a `List<Employee>` of `Employee` (abstract class) and store objects of `Salaried` and `Hourly` employees in it.

Interfaces

Interfaces

Are much like Abstract classes,
where class can extend only one
parent class, it can **implement**
more than one interfaces.

Its a keyword

Please note the special keyword implements to declare this class is a child of three interfaces

```
public class Person implements Comparable, Serializable, Cloneable, Runnable {  
  
    @Override  
    public void run() {  
  
    }  
}
```

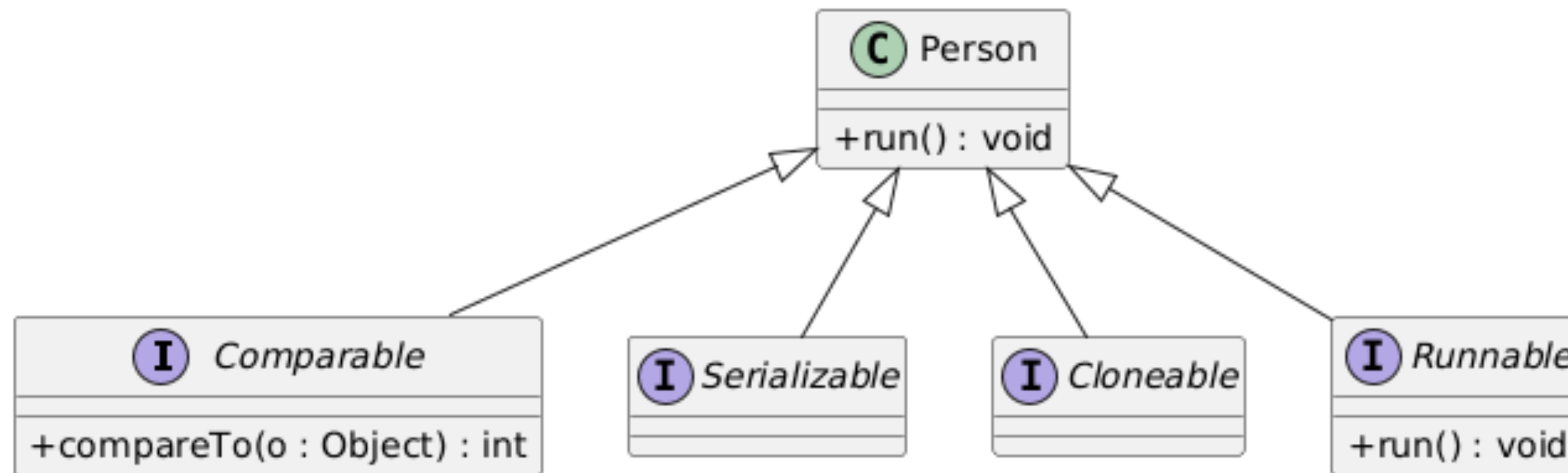
Implement all unimplemented methods in the child class

Only Runnable has an unimplemented run() method

Situation is exactly as of Abstract class so we have to implement interface methods

Class Diagram

```
public class Person implements Comparable, Serializable, Cloneable, Runnable {  
  
    @Override  
    public void run() {  
  
    }  
}
```



Tax Calculator

```
public interface TaxCalculator {  
    // implicitly public static final  
    double INCOME_LIMIT_1 = 50000;  
    double INCOME_LIMIT_2 = 250000;  
    double INCOME_LIMIT_3 = 500000;
```

Field are called shared constants

```
    // Tax rates corresponding to slabs  
    double TAX_RATE_1 = 0.02; // 2% for income between 50K and 2.5L  
    double TAX_RATE_2 = 0.05; // 5% for income between 2.5L and 5L  
    double TAX_RATE_3 = 0.10; // 10% for income above 5L  
    double calculateTax(double grossSalary);  
}
```

Exercise

Create an implementation of TaxCalculator & use it to calculate the net salary of salaried employee.

```
public class SlabedIncomeTax implements TaxCalculator {

    @Override
    public double calculateTax(double grossSalary) {
        if (grossSalary ≤ INCOME_LIMIT_1) {
            return 0;
        } else if (grossSalary ≤ INCOME_LIMIT_2) {
            return (grossSalary - INCOME_LIMIT_1) * TAX_RATE_1;
        } else if (grossSalary ≤ INCOME_LIMIT_3) {
            return (INCOME_LIMIT_2 - INCOME_LIMIT_1) * TAX_RATE_1 + (grossSalary -
INCOME_LIMIT_2) * TAX_RATE_2;
        } else {
            return (INCOME_LIMIT_2 - INCOME_LIMIT_1) * TAX_RATE_1
                + (INCOME_LIMIT_3 - INCOME_LIMIT_2) * TAX_RATE_2
                + (grossSalary - INCOME_LIMIT_3) * TAX_RATE_3;
        }
    }
}
```

Static

I am not associated with object rather with class

Static: Exercise

What if we want an automatic ID generation with object creation. Suppose you have two persons Student & Faculty, can we assign an ID on their object creation?

Exercise

```
public class Main {  
    public static void main(String[] args) {  
        int count=0;  
        Student ali = new Student("Ali");  
        Faculty zubair = new Faculty("Zubair");  
    }  
}
```

Try sending the count to Student and faculty classes & print the id using the toString() method.

Generate ID using Static

```
public abstract class Person {  
    private String name;  
    private int id;  
    static int count;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public Person(String name) {  
        this.name = name;  
    }  
    abstract public String getId();  
}
```

Notice the static field

Faculty class

```
public class Faculty extends Person {  
  
    public Faculty(String name) {  
        super(name);  
    }  
  
    @Override  
    public String getId() {  
        return ""+Person.id;  
    }  
}
```

Student class

```
public class Student extends Person {  
  
    public Student(String name) {  
        super(name);  
    }  
  
    @Override  
    public String getId() {  
        return ""+Person.id;  
    }  
}
```

But there is a problem here and also in the Person class.

Singleton Design Pattern

Singleton

Only one object should be created from a class.

Person class

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

Main class

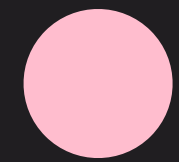
```
public class Main {  
  
    public static void main(String[] args) {  
        Person p = new Person("Zubair");  
  
    }
```

Constructors are designed for totally different functionality

Main class

```
public class Main {  
  
    public static void main(String[] args) {  
        Person p = new Person("Zubair");  
        Person u = new Person("Umaid");  
  
    }
```

Constructors are designed for totally different functionality



Make the constructor private

Person class

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

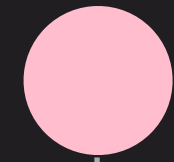
Person class

```
public class Person {  
    private String name;  
  
    private Person(String name) {  
        this.name = name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

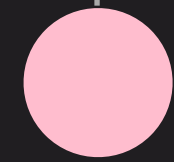
Main class

```
public class Main {  
  
    public static void main(String[] args) {  
        Person p = new Person("Zubair");  
  
    }  
}
```

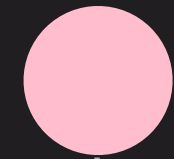
Constructor cannot be called in another class



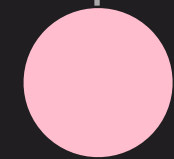
Make the constructor private



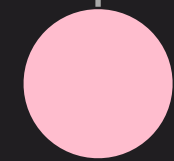
It can still be called internally within the class to create object



Make the constructor private



It can still be called internally within the class to create object



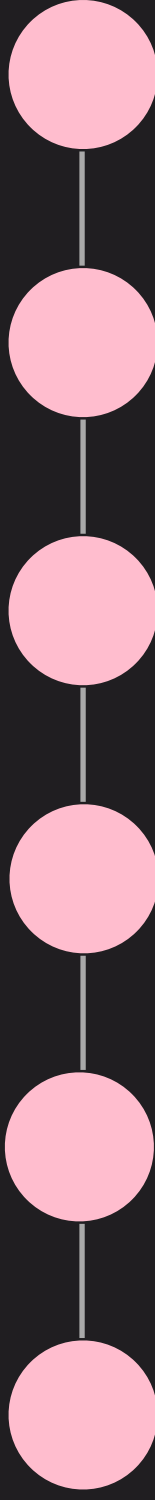
Make a Static method, since you still need to call class methods

Person class

```
public class Person {  
    private String name;  
  
    private Person(String name) {  
        this.name = name;  
    }  
    public static Person getInstance(String name){  
        return new Person(name);  
    }  
}
```

Returning object's reference like constructor

Note Static



Make the constructor private

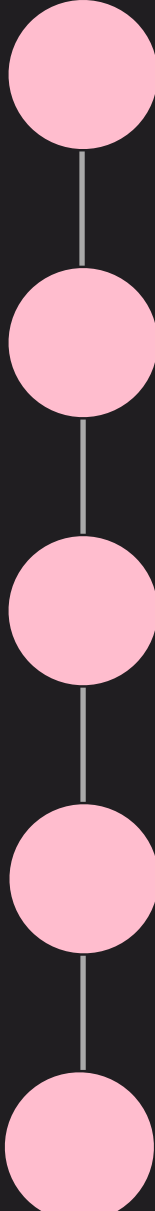
It can still be called internally within the class to create object

Make a Static method, since you still need to call class methods

That method will call the constructor to create its object

In that method, you should check if object is already created

For that you must have Static field in the class to store object reference



Make the constructor private

It can still be called internally within the class to create object

Make a Static method, since you still need to call class methods

That method will call the constructor to create its object

In that method, you should check if object is already created

Person class

```
public class Person {  
    private String name;  
  
    private Person(String name) {  
        this.name = name;  
    }  
  
}
```

Person class

```
public class Person {  
    private String name;  
    static Person person=null;  
    private Person(String name) {  
        this.name = name;  
    }  
}
```

```
}
```

Person class

```
public class Person {  
    private String name;  
    static Person person=null;  
    private Person(String name) {  
        this.name = name;  
    }  
    public static Person getInstance(String name){  
  
        return person;  
    }  
}
```

Person class

```
public class Person {  
    private String name;  
    static Person person=null;  
    private Person(String name) {  
        this.name = name;  
    }  
    public static Person getInstance(String name){  
        if(person ==null ) {  
            person = new Person(name);  
        }  
        return person;  
    }  
}
```

Person class

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

Main class

```
public class Main {  
  
    public static void main(String[] args) {  
        Person p = new Person("Zubair");  
  
    }  
}
```

Main class

```
public class Main {  
  
    public static void main(String[] args) {  
        Person p = Person.getInstance("Zubair");  
  
    }  
}
```

Main class

```
public class Main {  
  
    public static void main(String[] args) {  
        Person p = Person.getInstance("Zubair");  
        Person u = Person.getInstance("Umair");  
        System.out.println("Zubair"+ p +" Umair"+ u);  
  
    }  
}
```

>_ Zubair:com.example.Person@1dbd16a6 Umair:com.example.Person@1dbd16a6

Has-A Relationship

Address class

```
public class Address {  
    private String house;  
    private String street;  
    private String sector;  
  
    public Address(String house, String street, String sector) {  
        this.house = house;  
        this.street = street;  
        this.sector = sector;  
    }  
    //getters and setters...  
}
```

Student class

```
public class Student extends Person{  
    private String regNo;  
    private Address address;  
  
    public Student(String name, String regNo, Address address) {  
        super(name);  
        this.regNo = regNo;  
        this.address = address;  
    }  
  
    public Address getAddress() {  
        return address;  
    }  
  
    public void setAddress(Address address) {  
        this.address = address;  
    }  
}
```

Expecting an object here

Main class

```
public class Main {  
    public static void main(String[] args) {  
  
        Address homeAddress = new Address("60", "25", "J");  
        Student zubair = new Student("ms970130", homeAddress);  
    }  
}
```

Object passed to the Student constructor

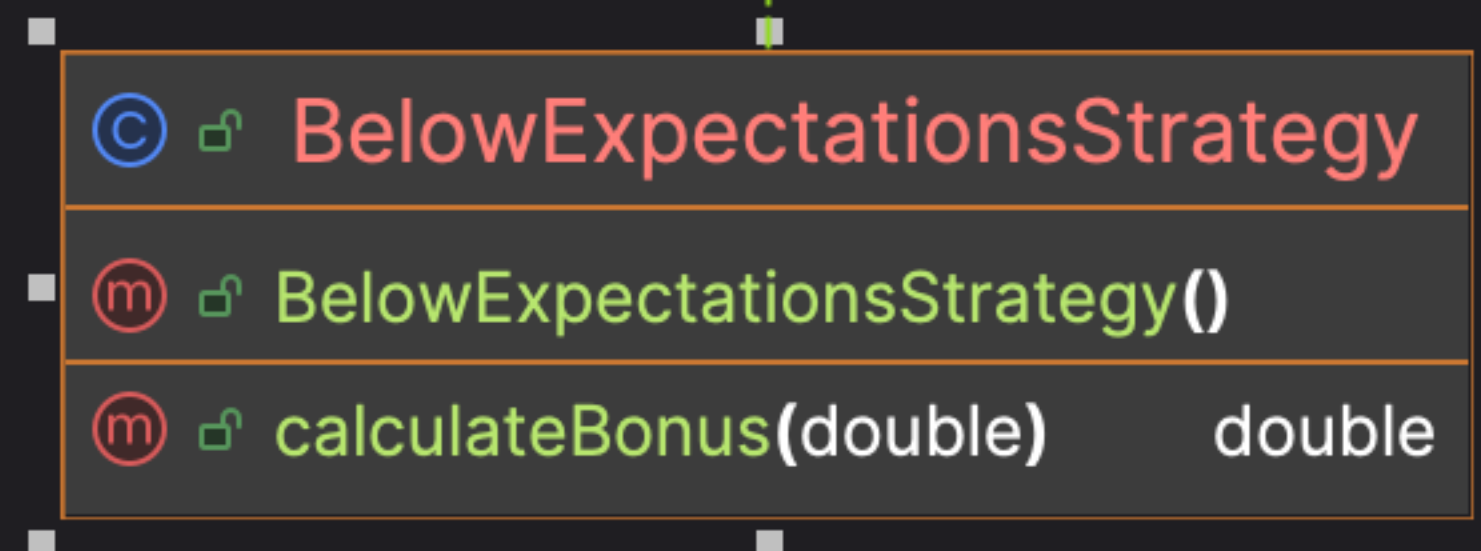
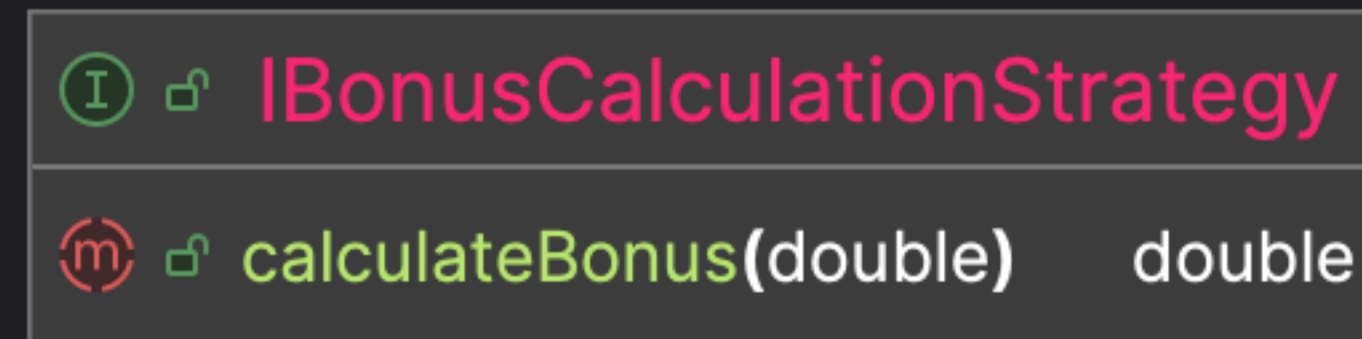
Strategy Design Pattern

Bonus Calculation

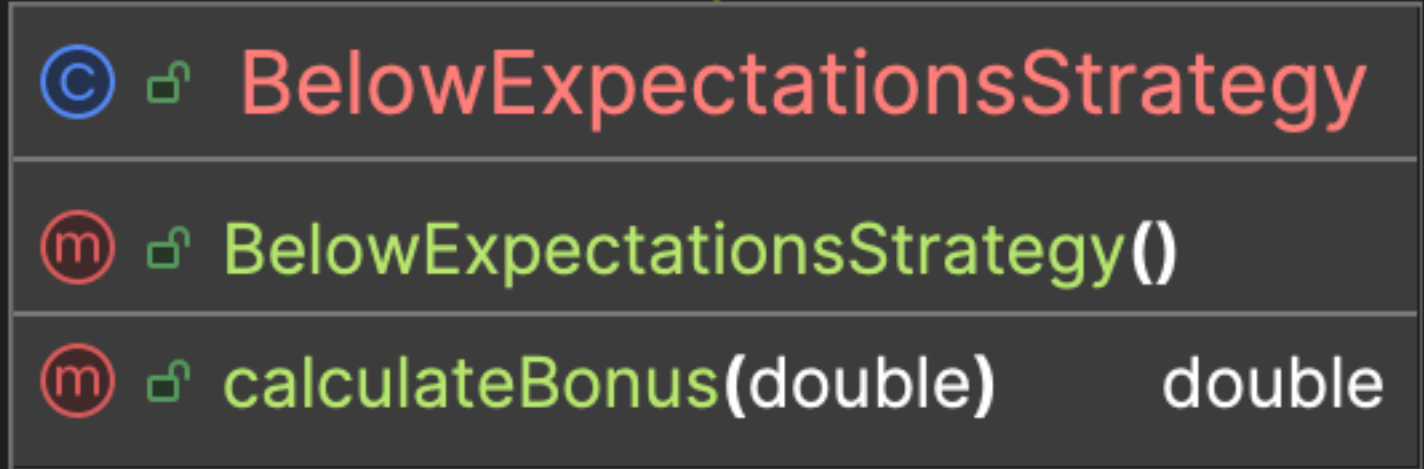
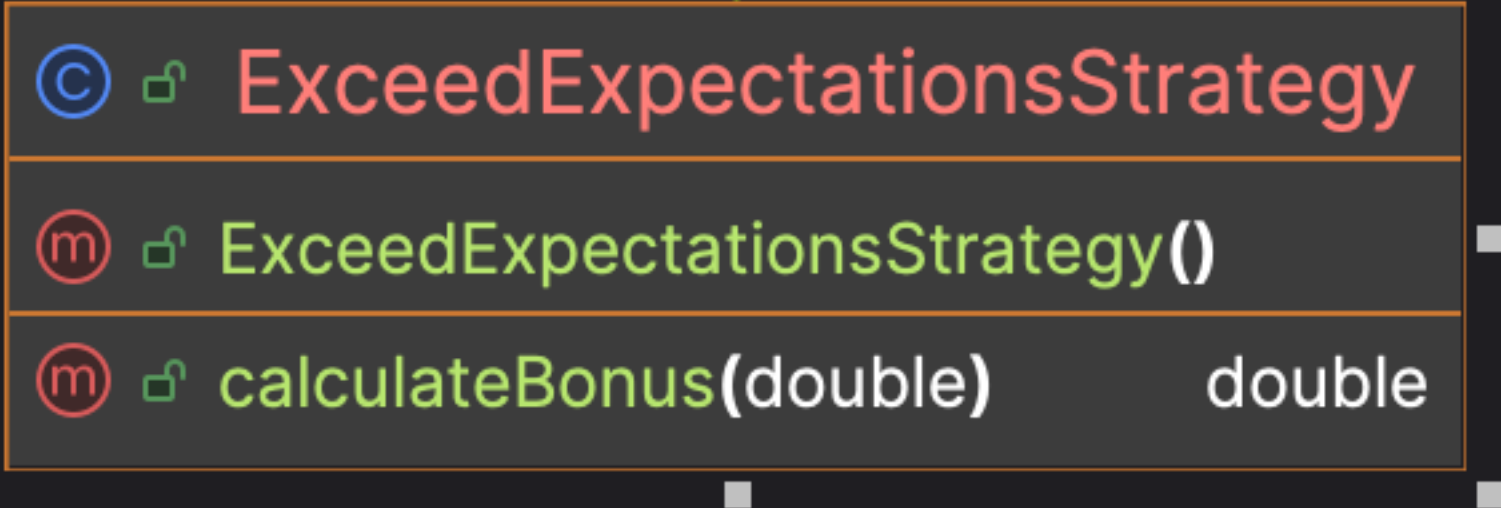
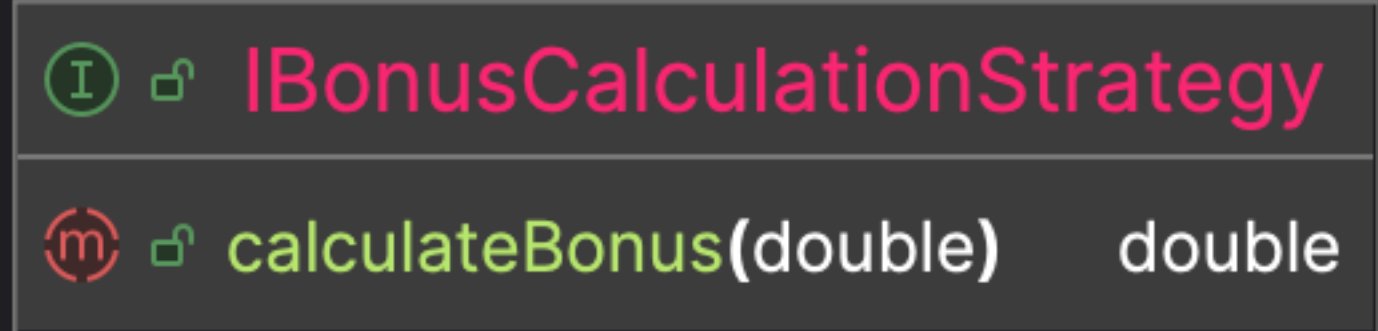
A firms Employees bonus calculation using Strategy Design Pattern.

```
public interface IBonusCalculationStrategy {  
    double calculateBonus(double baseSalary);  
}
```

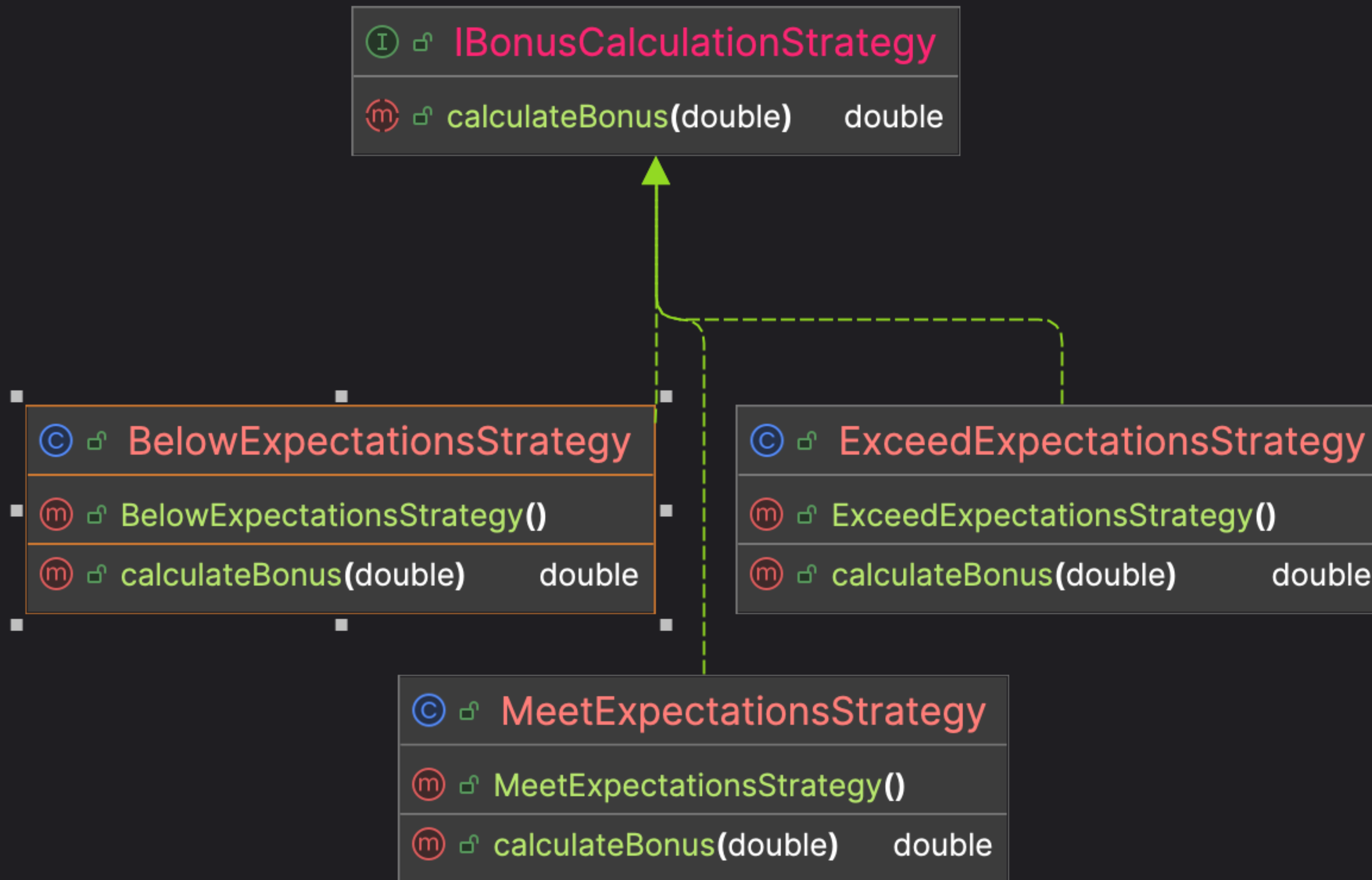
```
public interface IBonusCalculationStrategy {  
    double calculateBonus(double baseSalary);  
}  
  
public class BelowExpectationsStrategy implements  
IBonusCalculationStrategy{  
    @Override  
    public double calculateBonus(double baseSalary) {  
        return baseSalary * 0.05;  
    }  
}
```



```
public interface IBonusCalculationStrategy {  
    double calculateBonus(double baseSalary);  
}  
  
public class ExceedExpectationsStrategy implements  
IBonusCalculationStrategy{  
    @Override  
    public double calculateBonus(double baseSalary) {  
        return (baseSalary * 0.20) + 1000;  
    }  
}
```



```
public interface IBonusCalculationStrategy {  
    double calculateBonus(double baseSalary);  
}  
  
public class MeetExpectationsStrategy implements  
IBonusCalculationStrategy{  
    @Override  
    public double calculateBonus(double baseSalary) {  
        return baseSalary * 0.10;  
    }  
}
```



```
public class Employee {  
    private String name;  
    private double baseSalary;  
    private IBonusCalculationStrategy bonusStrategy;  
}
```

```
public class Employee {  
    private String name;  
    private double baseSalary;  
    private IBonusCalculationStrategy bonusStrategy;  
  
    public Employee(String name, double baseSalary,  
IBonusCalculationStrategy bonusStrategy) {  
        this.name = name;  
        this.baseSalary = baseSalary;  
        this.bonusStrategy = bonusStrategy;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Employee employee1 = new Employee(  
            "Ali",  
            50000,  
            new MeetExpectationsStrategy()  
        );  
    }  
}
```

```
System.out.println(employee1.calculateYearEndBonus());
```

```
>_ 5000.0
```

```
public class Main {  
    public static void main(String[] args) {  
        Employee employee2 = new Employee(  
            "Atif",  
            50000,  
            new ExceedExpectationsStrategy()  
        );  
    }  
}
```

```
System.out.println(employee2.calculateYearEndBonus());
```

```
>_ 11000.0
```

```
public class Main {  
    public static void main(String[] args) {  
        Employee employee3 = new Employee(  
            "Akif",  
            50000,  
            new BelowExpectationsStrategy()  
        );  
    }  
}
```

```
System.out.println(employee3.calculateYearEndBonus());
```

```
>_ 2500.0
```


JDBC

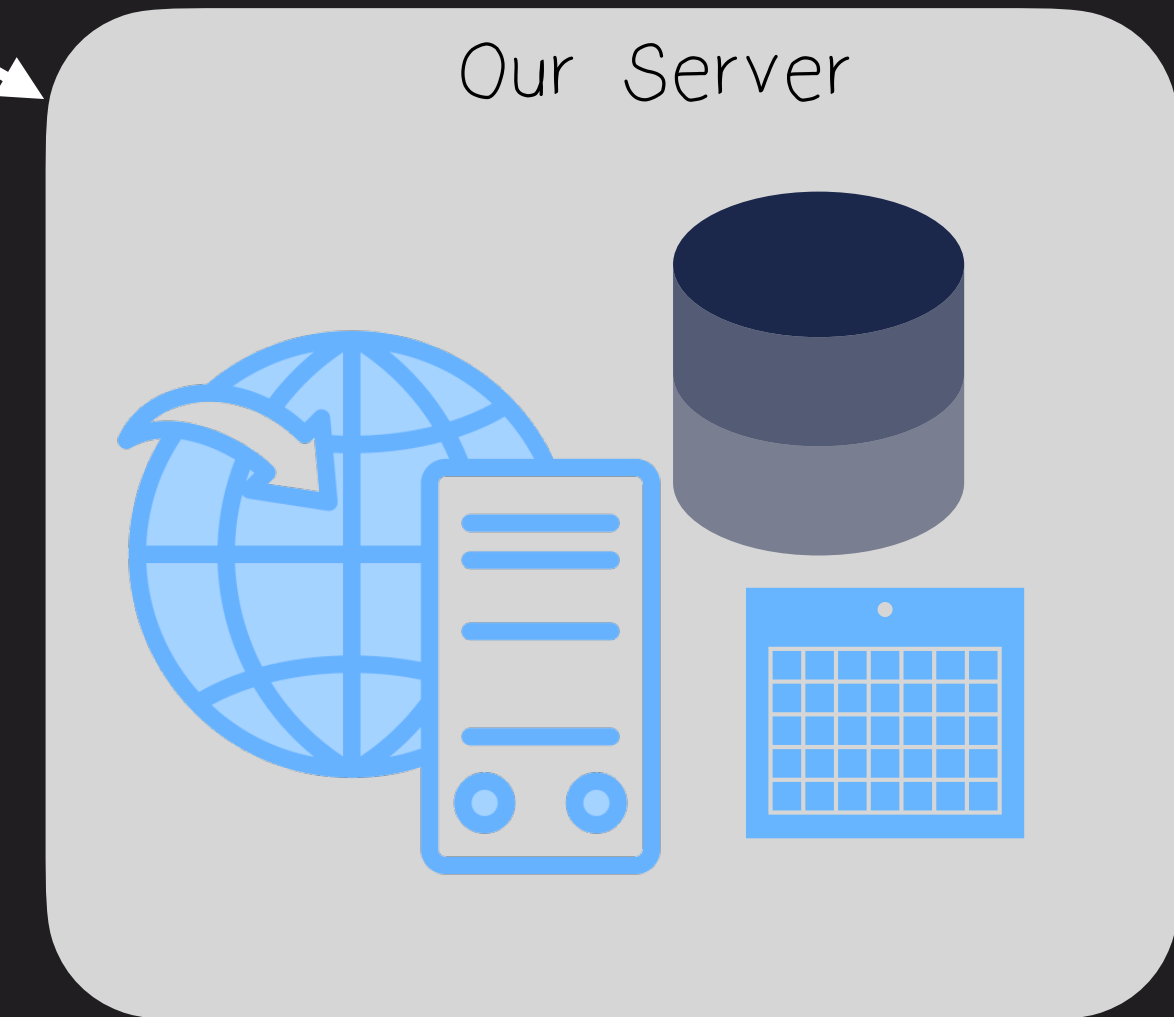
What is JDBC ?

JDBC is an API that helps applications to communicate with databases. It allows Java programs to connect to a database, run queries, retrieve and manipulate data. Using JDBC, application can connect to different database e.g. Postgres, Oracle MySql etc.

**Database on the Server
machine**

request

response



```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/userdb","zubair","123");

            String query = "select * from users";
            ResultSet rs = con.createStatement().executeQuery(query);
            while (rs.next()) {
                System.out.println(rs.getInt("id"));
                System.out.println(rs.getString("name"));
                System.out.println(rs.getString("password"));
            }

        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

```
import java.sql.*;  
public class Main {  
    public static void main(String[] args) {
```

```
        Class.forName("org.postgresql.Driver");  
    }  
}
```



Its a unsafe method

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {

        Class.forName("org.postgresql.Driver");
```



You have to use try and catch

```
    }
}
```

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        }
    }
}
```

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            System.out.println("a error occurred");
        }
    }
}
```

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
    }
}
```



```
}
```

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Step

1

JVM should exit with an exception

}

}

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/userdb","zubair","123");

        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Step 1

Step 2

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/userdb","zubair","123");
            String query = "select * from users";

        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Step 1

Step 2

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/userdb","zubair","123");
            String query = "select * from users";
            ResultSet rs = con.createStatement().executeQuery(query);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Step 1

Step 2

Step 3

```
import java.sql.*;
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("org.postgresql.Driver");
            System.out.println("Drivers loaded");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);
        }
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/userdb","zubair","123");
            String query = "select * from users";
            ResultSet rs = con.createStatement().executeQuery(query);
            while (rs.next()) {

            }

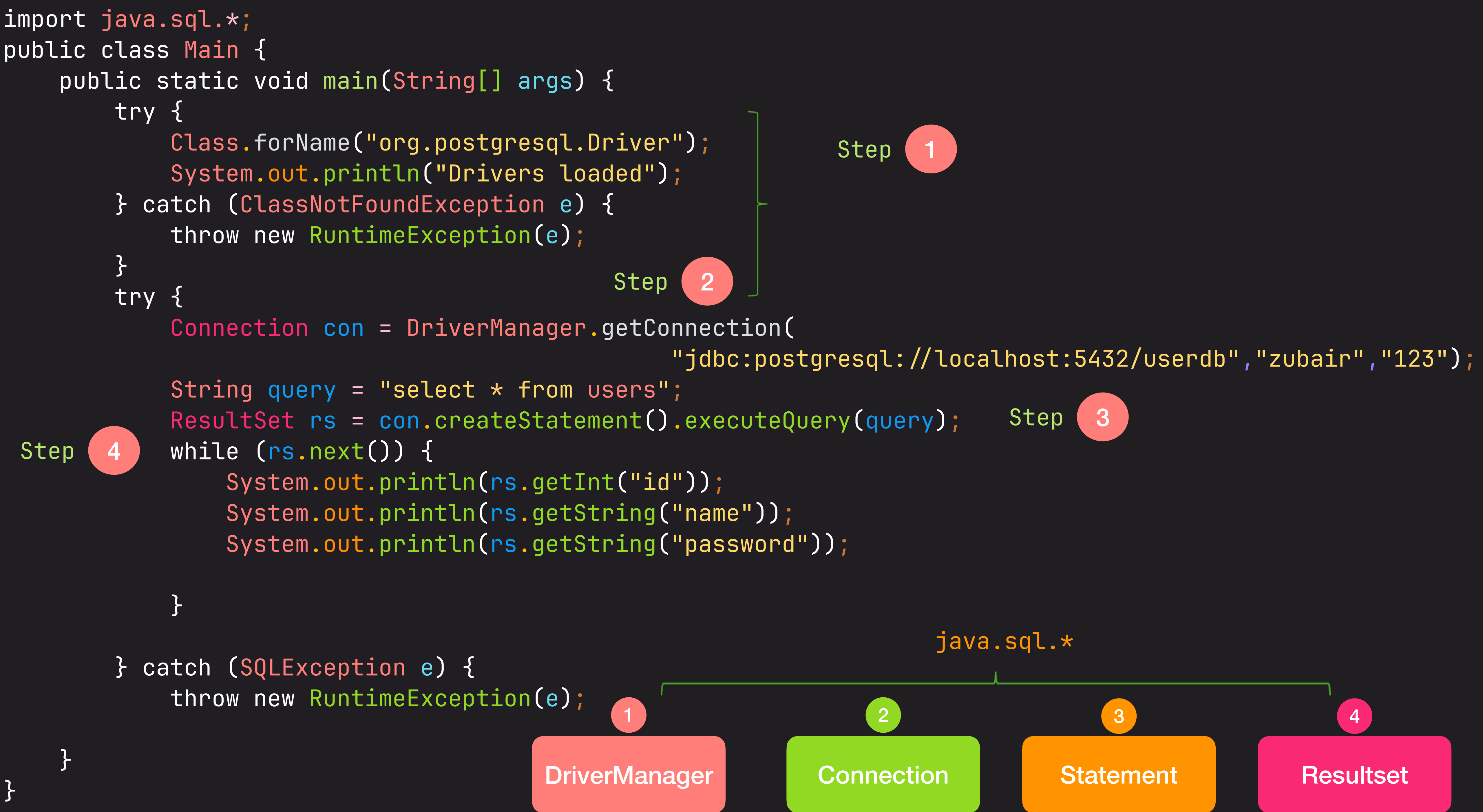
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Step 1

Step 2

Step 3

Step 4



DAO Design Pattern

What is DAO ?

Data Access Objects is a structural pattern used to separate the data access logic from business logic. This forms the data layer that the business layer can consult to get data from the database.

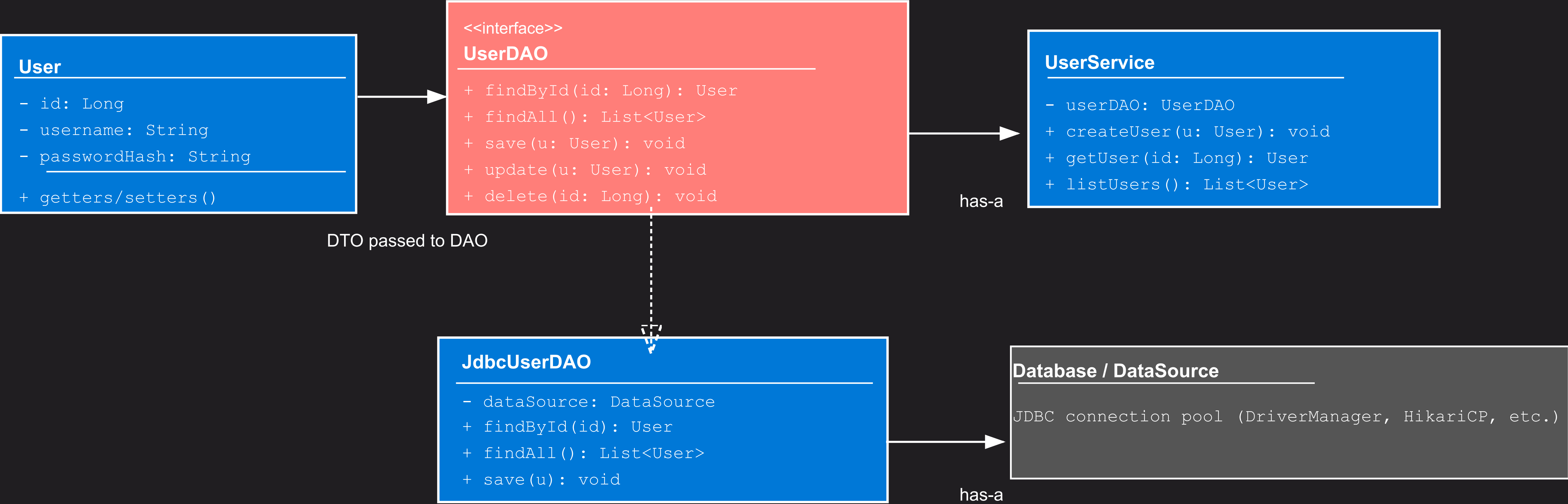
Components

DAO Components

- Model/Entity Data Transfer Objects
- DAO Interface
- DAO Concrete Class
- Data Source/Factory

DAO (Data Access Object) — Pattern Overview

Separate persistence (DAO) from business logic (Service). Entity objects (DTO) move between layers.



Legend

- User — entity / DTO (simple data holder)
- UserDao — interface defining persistence operations
- JdbcUserDAO — concrete DAO implementing UserDao
- UserService — business layer that uses DAO

UserDAO: Example

User table

user

PK ID	— integer, primary key
username	— varchar(255)
password	— varchar(255) (hashed)

Tip: never store plaintext passwords — store salted hashes instead.

DAO Components

- Model/Entity Data Transfer Objects
- DAO Interface
- DAO Concrete Class
- Data Source/Factory

Model

Model / Entity / DTO

```
public class User {  
    private int id;  
    private String username;  
    private String password;  
  
    public User(int id, String username, String password) {  
        this.id = id;  
        this.username = username;  
        this.password = password;  
    }  
  
    // getter & setters  
  
}
```

DAO Interface

DAO

```
public class UserDao {  
    private Connection connection;  
  
    public UserDao() {  
        this.connection = DbUtil.getConnection();  
    }  
  
    public void addUser(User user) {}  
  
    public void deleteUser(int userId) {}  
  
    public void updateUser(User user) {}  
  
    public List<User> getAllBooks() {}  
  
    public User getUserById(int userId) {}  
  
}
```

DAO

```
public interface UserDao {  
  
    public UserDao() {  
        this.connection = DbUtil.getConnection();  
    }  
  
    public void addUser(User user) {}  
  
    public void deleteUser(int userId) {}  
  
    public void updateUser(User user) {}  
  
    public List<User> getAllBooks() {}  
  
    public User getUserById(int userId) {}  
  
}
```

DAO

```
public interface UserDao {  
  
    public void addUser(User user) {}  
  
    public void deleteUser(int userId) {}  
  
    public void updateUser(User user) {}  
  
    public List<User> getAllBooks() {}  
  
    public User getUserById(int userId) {}  
  
}
```

DAO

```
public interface UserDao {  
  
    public void addUser(User user);  
  
    public void deleteUser(int userId) {}  
  
    public void updateUser(User user) {}  
  
    public List<User> getAllBooks() {}  
  
    public User getUserById(int userId) {}  
  
}
```

DAO

```
public interface UserDao {  
  
    public void addUser(User user);  
  
    public void deleteUser(int userId);  
  
    public void updateUser(User user);  
  
    public List<User> getAllBooks();  
  
    public User getUserById(int userId);  
  
}
```

DAO Interface

```
public interface UserDao {  
  
    public void addUser(User user);  
  
    public void deleteUser(int userId);  
  
    public void updateUser(User user);  
  
    public List<User> getAllBooks();  
  
    public User getUserById(int userId);  
  
}
```

UserDAO

UserDAOImpl

DAOImpl

```
public class UserDaoImpl implements UserDao {  
    private Connection connection;  
  
    public UserDaoImpl() {  
        this.connection = DbUtil.getConnection();  
    }  
}
```

DAOImpl

```
public class UserDaoImpl implements UserDao {
    private Connection connection;

    public UserDaoImpl() {
        this.connection = DbUtil.getConnection();
    }

    public void addUser(User user) {
        try {
            PreparedStatement preparedStatement = connection.prepareStatement(
                "insert into users(username, password) values (?, ? )");
            // Parameters start with 1
            preparedStatement.setString(1, user.getUsername());
            preparedStatement.setString(2, user.getPassword());
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

AddUser

```
public class UserDaoImpl implements UserDao {
    private Connection connection;

    public UserDaoImpl() {
        this.connection = DbUtil.getConnection();
    }

    public void addUser(User user) {
        try {
            PreparedStatement preparedStatement = connection.prepareStatement(
                "insert into users(username, password) values (?, ? )");
            // Parameters start with 1
            preparedStatement.setString(1, user.getUsername());
            preparedStatement.setString(2, user.getPassword());
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

DeleteUser

```
public class UserDaoImpl implements UserDao {
    private Connection connection;

    public UserDaoImpl() {
        this.connection = DbUtil.getConnection();
    }
    public void deleteUser(int userId) {
        try {
            PreparedStatement preparedStatement = connection.prepareStatement(
                "delete from users where id=?");
            // Parameters start with 1
            preparedStatement.setInt(1, userId);
            preparedStatement.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

UpdateUser

```
public class UserDaoImpl implements UserDao {
    private Connection connection;

    public UserDaoImpl() {
        this.connection = DbUtil.getConnection();
    }
    public void updateUser(User user) {
        try {
            PreparedStatement preparedStatement = connection.prepareStatement(
                "update users set username=?, password=? where id=?");
            // Parameters start with 1
            preparedStatement.setString(1, user.getUsername());
            preparedStatement.setString(2, user.getPassword());
            preparedStatement.setInt(3, user.getId());
            preparedStatement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

GetUserById

```
public class UserDaoImpl implements UserDao {  
    // Omitted Constructor  
  
    public User getUserById(int userId) {  
        User user = new User();  
        try {  
            PreparedStatement preparedStatement = connection.  
                prepareStatement("select * from users where id=?");  
            preparedStatement.setInt(1, userId);  
            ResultSet rs = preparedStatement.executeQuery();  
  
            if (rs.next()) {  
                user.setId(rs.getInt("id"));  
                user.setUsername(rs.getString("username"));  
                user.setPassword(rs.getString("password"));  
            }else{  
                user=null;  
            }  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

GetAllUsers

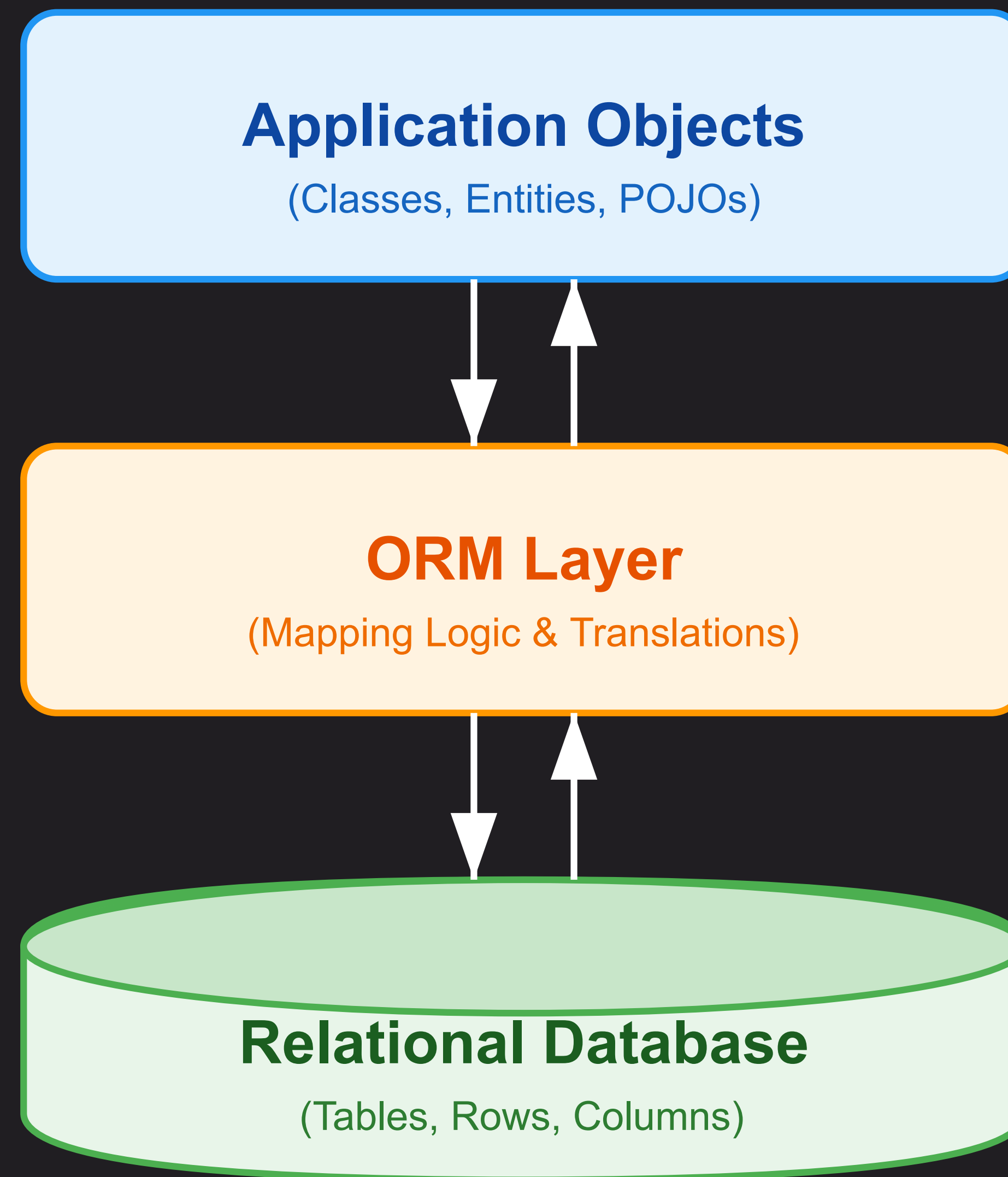
```
public class UserDaoImpl implements UserDao {
    private Connection connection;

    public UserDaoImpl() {
        this.connection = DbUtil.getConnection();
    }
    public List<User> getAllUsers() {
        ResultSet rs = null;
        List<User> users = new ArrayList<User>();
        try {
            Statement statement = connection.createStatement();
            rs = statement.executeQuery("select * from users");
            while (rs.next()) {
                User user = new User();
                user.setId(rs.getInt("id"));
                user.setUsername(rs.getString("username"));
                user.setPassword(rs.getString("password"));
                users.add(user);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try { rs.close(); } catch (SQLException e) { throw new RuntimeException(e); }
            // close resultset
            // close statement
            // close connection in the case of connection pooling
        }

        return users;
    }
}
```

Object Relational Mapping

ORM



Feature	Role Attribute	Object Privilege
What it is	A global capability or setting for a role defined at the cluster level.	A specific permission to perform an action on a particular database object.
Scope	Applies to the role across the entire database cluster.	Specific to a single database or schema. Inherited based on object.
Inheritance	Generally not inherited by member roles. Ex: SUPERUSER, LOGIN not inherited.	Inherited by member roles by default.
Example	LOGIN, SUPERUSER, CREATE DATABASE, CREATEROLE.	SELECT, INSERT, UPDATE, DELETE, EXECUTE, USAGE.

Feature	Role Attribute	Object Privilege
What it is	A global capability or setting for a role defined at the cluster level.	A specific permission to perform an action on a particular database object.
Scope	Applies to the role across the entire database cluster.	Specific to a single database object. Inherited based on object.
Inheritance	Generally not inherited by member roles. Ex: SUPERUSER, LOGIN not inherited.	Inherited by member roles by default.
Example	LOGIN, SUPERUSER, CREATE, CREATE ROLE, CREATE DATABASE, CREATE SCHEMA, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE, CREATE FUNCTION, CREATE PROCEDURE, CREATE TRIGGER, CREATE POLICY, CREATE ROLE, CREATE USER, CREATE GROUP, CREATE ROLE, CREATE USER, CREATE GROUP, CREATE ROLE, CREATE USER, CREATE GROUP.	SELECT, INSERT, UPDATE, DELETE, EXECUTE, USAGE.

